

BOOTCAMP MLOps

BOOTCAMP MLOps:

What we will learn to day:
From Notebooks to Production

Danh Pham

Goals

- Structured Foundation 🏗️: Bridge model development and reliable operational deployment with MLOps.
- Isolate & Manage Dependencies 📦: Solve "dependency hell" using virtual environments.
- Introduce Version Control 💾: Use Git for tracking changes and GitHub for collaboration (add → commit → push workflow).

Contents

What is MLOps?

The problem with Notebooks for Production

The Solution: Environments, Code Structure, and Version Control

Hands-on Lab

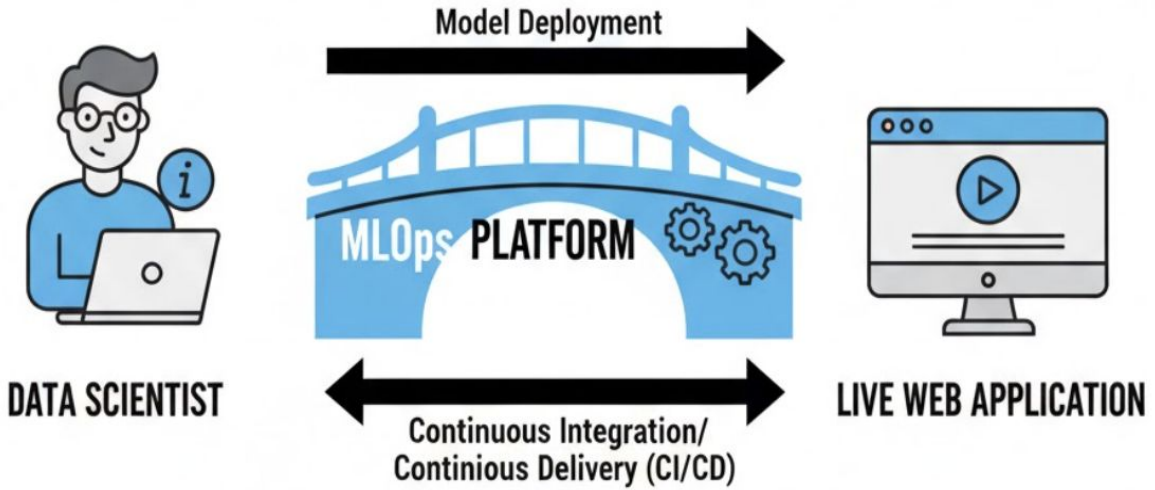
COLE

What is MLOps? (And why should we care?)

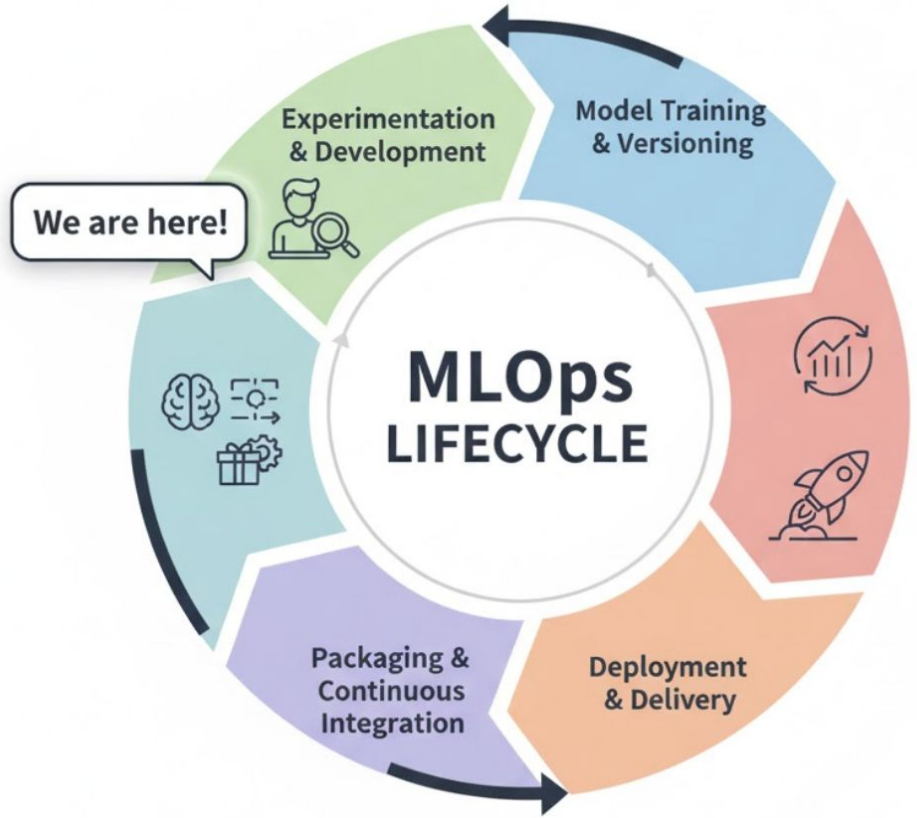
MLOps is a set of practices that aims to deploy and maintain machine learning models in production reliably and efficiently.

MLOps solves problems

- Slow/Manual deployment.
- Models management.
- Models monitoring and improvement.



The Journey of a model

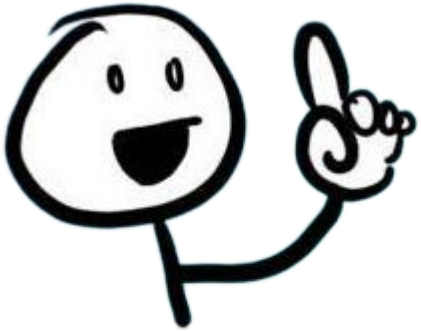


Research vs. Production

	Research / Exploration	Production / Operation
Goal	Speed of discovery, visullization	Reliability, automation, scalabilty
Tool	Jupeter Notebooks (Excellent!)	Python Scripts (Essential!)
Output	Insights, charts, a protdype model	A robost, testable software component

COLE

The Danger of a "Hidden State"



```
[5]: x = 10

[6]: print(f"x = {x}")

     x = 10
```



```
[5]: x = 10

[8]: print(f"x = {x}")

     x = 5

[7]: x = 5
```

COLE

Why Git and Notebooks Don't Mix Well

```
100       "outputs": [
101        {
102         "data": {
103 -         "image/png":
"iVBORw0KGgoAAAANSUhEUgAAAXYAAAD/CAYAAADllv3BAAAABHNCSVQICAgIfAhkiAAAAAlwSF
lz\nAAALEgAACxIB0t1+/AAAIABJREFUeJzt3XlUFGe6P/BvASIgKDpHZ4JxQRLGCTJGMMEljKI
SnaBG\nwE5wQaMTl8TrGFACRNlDUMHlGsSIGoyaEReIcZh71RjUmaCiQiCgGZwfIiqYG0lAGhob
kPf3B3ZL\ny9IL1V3VzfM5x3Okmq7+dlk+vLxd9bwcY4yBEEKIyTATOgAhhBB+UWEnhBBTQgAAA
EJMDBV2Qggx\nMVTYCSHExFBhJ4QQE6NRYS8sLERgYGC77dnZ2Zg7dy4CAgJw7Ngx3sMRQgjRno
W6b9i7dy++/vpr\n9OnTR2V7c3MzNm7ciMzMTPTu3Rvz5s3D1KlTMWDAAL2FJYQQop7aEfuwYcO
wc+fOdttLS0sxbNgw\n2NraolevXnB3d8fVq1f1EpIQQojm1BZ2b29vmJubt9teV1cHOzs75dd9
+vSBVCrlNx0hhBCt6fzh\nqa2tLerq6pRf19fXo2/fvryEIoQQoju1c+wKz7aUcXJyQnl5OWpra
2FlZYWrV6/iL3/5S4fPzcvL\n615KQgjpodzd3bV+jsaFneM4AEBWVhYaGhogkUgQHh6OpUuXgj
EGiUSCQYMG8RpOnyorK+Hg4CB0\njHbEmIsyaYYyaU6MucSYSddBsUaFffDgwUhPTwcAzJw5U7l
98uTJmDx5sk4vTAghRD/oBiVCCDEx\nVNgJIcTEaDzHTkxHWVk5IiL2o6KiBYMHmyEu7h04Og4T
OhYhhCdU2DvR2NiI9z76CLs++QSWlpZC\nx+FNWVk5vL0/RWlpDIA+AOpx+XIUvvlmNRV3QkwET
cV0YllEBL4YPBjLIyOFjsKriIj9bYo6APRB\naWkMIiL2C5iKEMInKuwd+PzoUXxtb4/HY8bgRN
++SDOhBmcVFS14WtQV+qCyskWIOIQQPaDC/ozS\nW7cQl52Nh+PHAwAeTpiA2G+/RemtWwIn48f
gwWYA6p9uGPcL0OchHByengo1TU34xy+/GD4cIYQX\nVNif8cGmTbj95psq227Pno0PNm0SKBG/
4uLegZNTFJTFvcgCfYO/QkhMa1vmmqYmrC8rw0RqD0GI\n0aLC/oztoaEY/vXXKtuGnzyJ7aGhA
```

```
94        "outputs": [
95         {
96          "data": {
97 +         "image/png":
"iVBORw0KGgoAAAANSUhEUgAAAW4AAAD1CAYAAABwdB+7AAAABHNCSVQICAgIfAhkiAAAAAlwSF
lzAAALEgAACxIB0t1+/AAAADl0RVh0U29mdHdhcmUAbWF0cGxvdGxpYiB2ZXJzaW9uIDIuMS4yL
CBodHRwOi8vbWF0cGxvdGxpYi5vcmcvNQv5yAAAIABJREFUeJzt3Xl8VPW9//HXXSMQZMU
ZFG/pAYVIxBEQRQXKlQMVIv9RbBYpe5wFVBcan91K1exYq1tEeG2WhsxUgEvVAKi0pJgCja5hq9
gLgSIkCBLQsw2k7l/TGachMk6Z2bOJJ/n4+FD5pyZc96T5ZPvfM85n2NxOBwIEkKIQ6QmJiYmJ
J0iOTk5MSEhIS0tLT09PSMjIzMzMysrKzs7Oycnp6CgoKioqLi4uLS0tKysrLKysqKiourq6pqa
mtra2vr6+sbGxqampubm5tbW1vb29s7Ozu7u7t7e3v7+/sHBweHh4dHR0fHx8cnJyenp6dnZ2fn
5+cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dX
V1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeX
l5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cX
FxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX19cXFxeXl5dXV1fX
19
```

It Works On My Machine… But…





Or maybe not

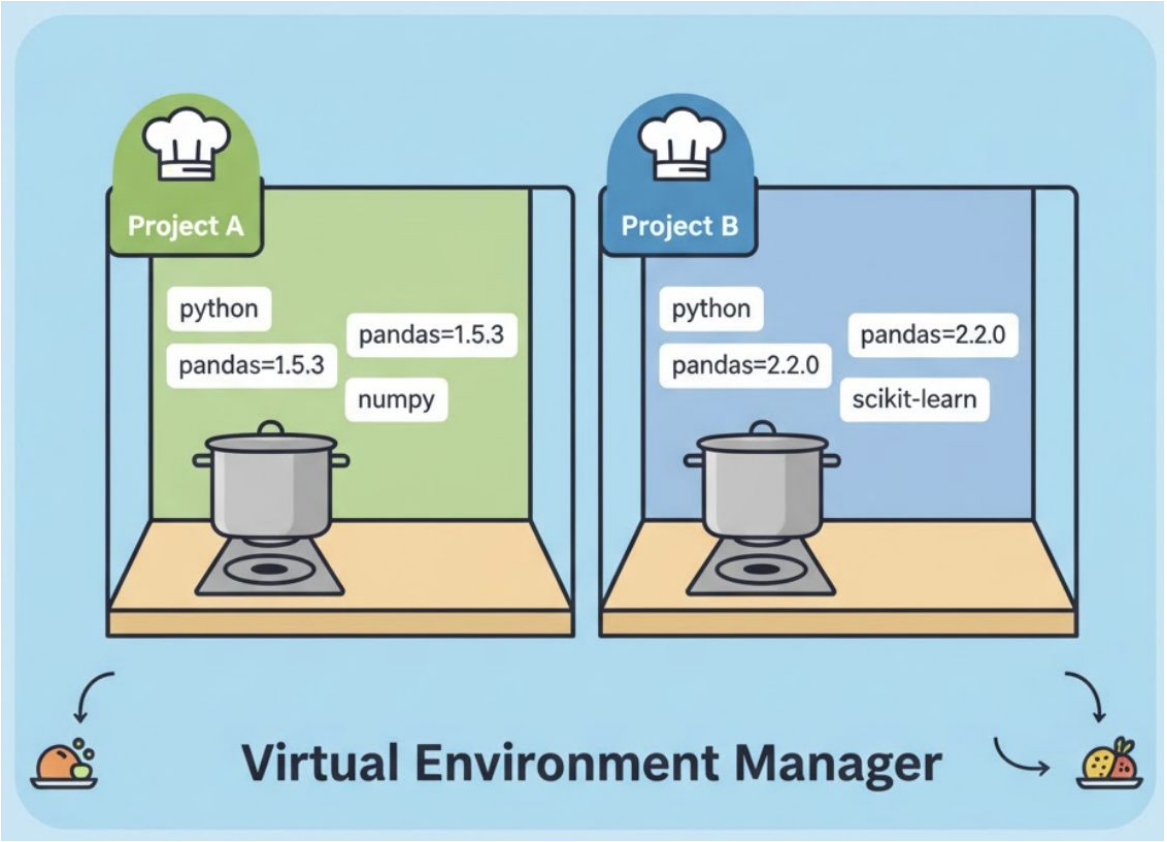
COLE

The Solution: Virtual Environments with venv

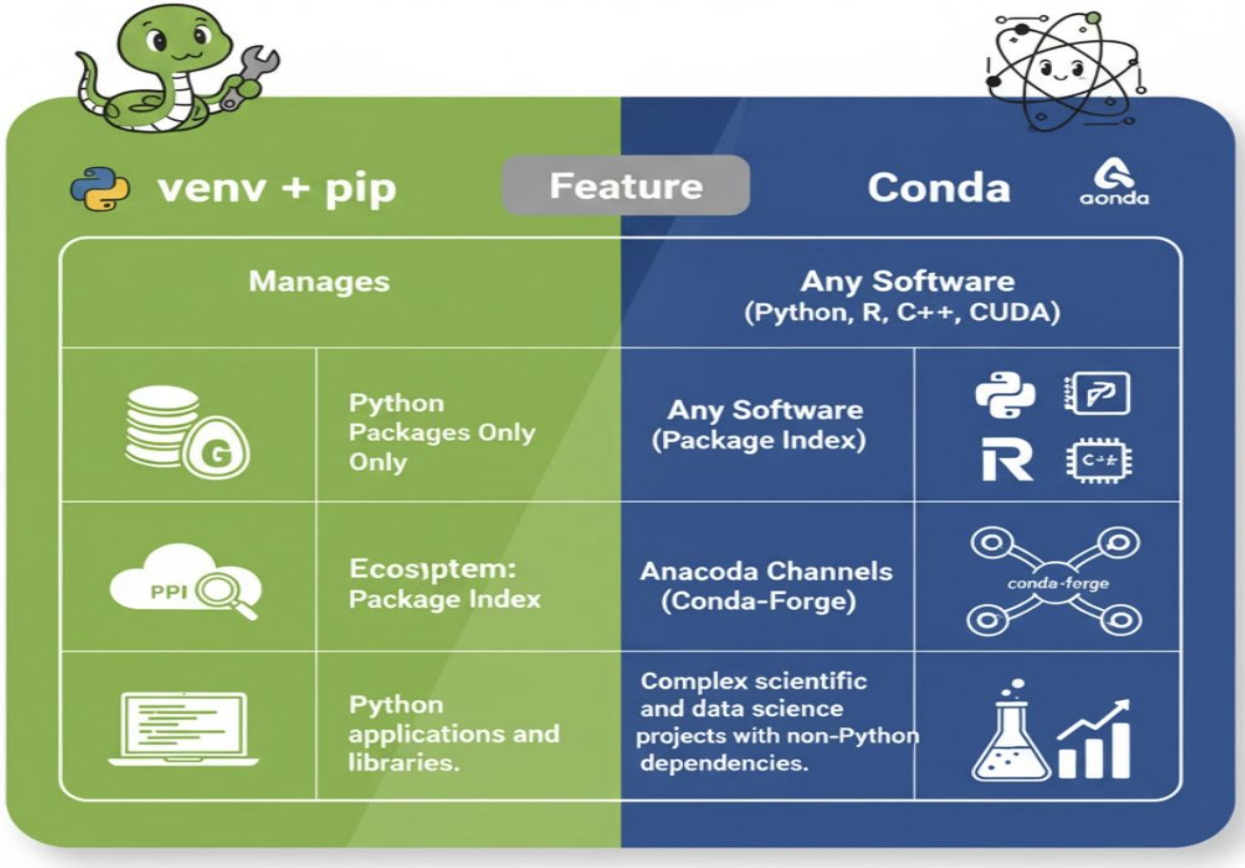
A *venv* creates an isolated directory with a specific Python interpreter and its supporting files, separate from the main system's Python and other virtual environments.

Key commands:

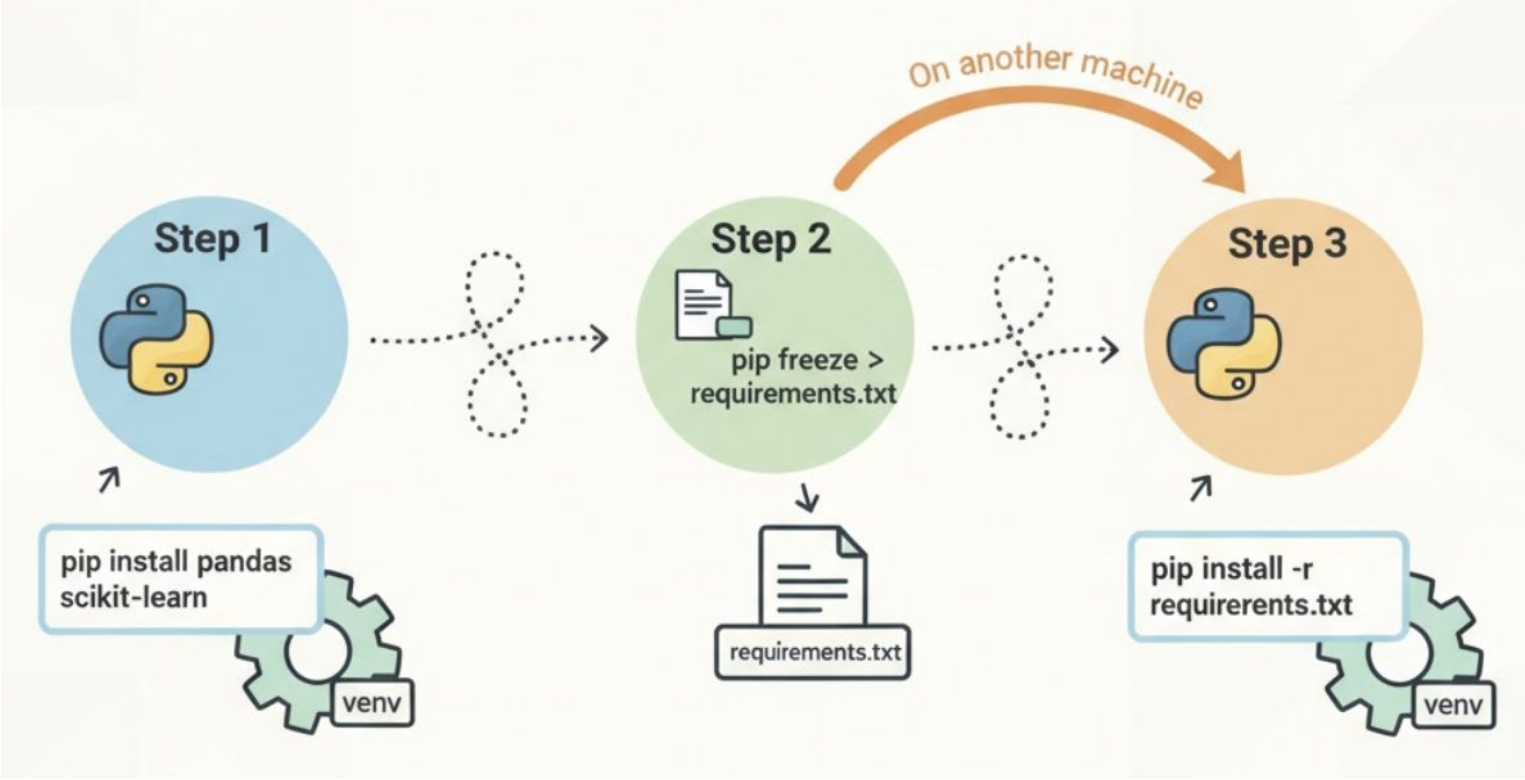
```
python -m venv .venv
source
.venv/bin/activate
```



Venv vs. Conda: What's the Difference?



Dependency Management: The Classic Approach



Dependency Management: The Modern Approach (Poetry)

Key commands:

```
pip install poetry

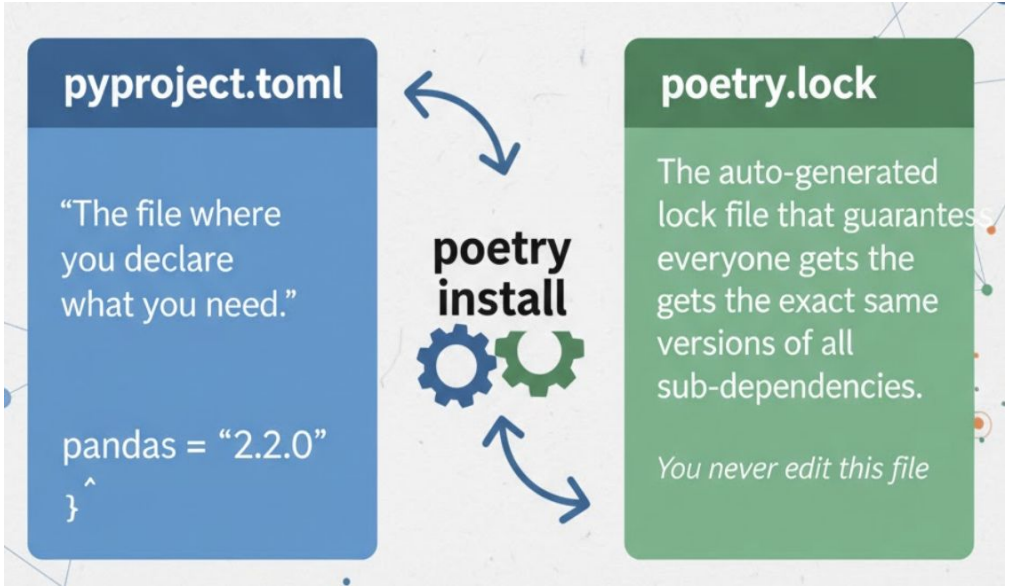
poetry new <my-package>

poetry add <lib1> <lib2>

poetry lock

poetry install

poetry run <command>
```



COLE

https://python-poetry.org/docs/cli/

Dependency Management: The Need for Speed (uv)

Key commands:

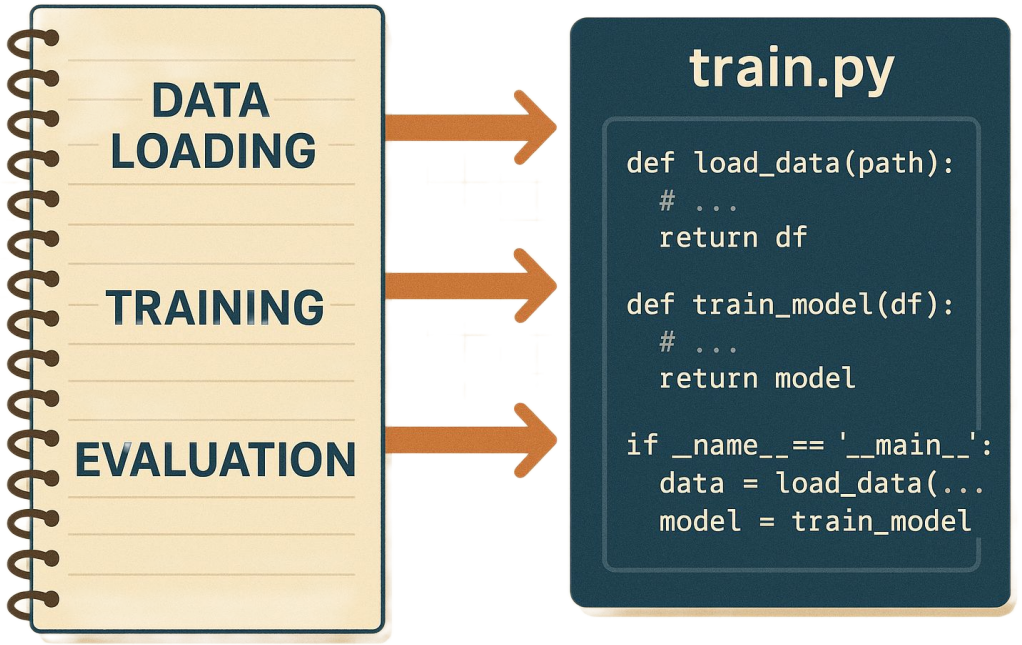
`uv init`: create a new project
`uv add`: add dependencies
`uv lock`: update project's lock file
`uv venv`: create a virtual environment
`uv help`: Display documentation for a command



https://docs.astral.sh/uv/reference/cli/

The Solution: Structure Your Code

```
ml-project/
├── .venv/              # Your isolated virtual environment
├── data/               # Raw and processed datasets
├── notebooks/          # Your original, exploratory notebooks
├── src/
│   └── train.py        # Your production training script
├── tests/              # Your unit tests
├── .gitignore          # Tells Git what files to ignore
└── requirements.txt    # Your project's dependencies
```



A Clean and Standard Project Layout

Git is like having an infinite 'undo' button for your entire project, with a detailed log of every 'save'.

GitHub is the cloud drive where you store and share those saved versions.



[master] 6c6faa5 My first commit - John Doe

[develop] 3e89ec8 Develop a feature - part 1 - John Doe

[develop] e188fa9 Develop a feature - part 2 - John Doe

[master] 665003d Fast bugfix - John Fixer

[myfeature] eaf618c New cool feature - John Feature

[master] 8f1e0e7 Merge branch `develop` into `master` - John Doe

[master] 6a3dacc Merge branch `myfeature` into `master` - John Doe

0.1

[master] abcdef0 Release of version 0.1 - John Releaser

COLE

The Fundamental Git Workflow

Key commands:

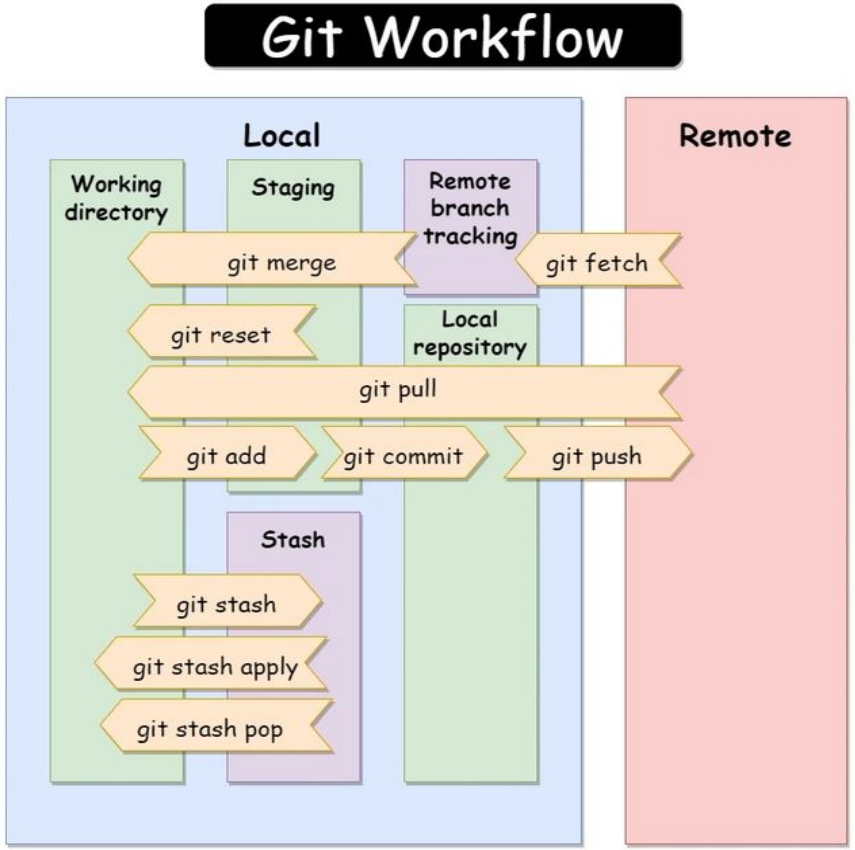
`git clone`: clone (download) code from cloud repository (remote repo).

`git add`: add changes from working dir to staging.

`git commit`: save changes to local repo with a message.

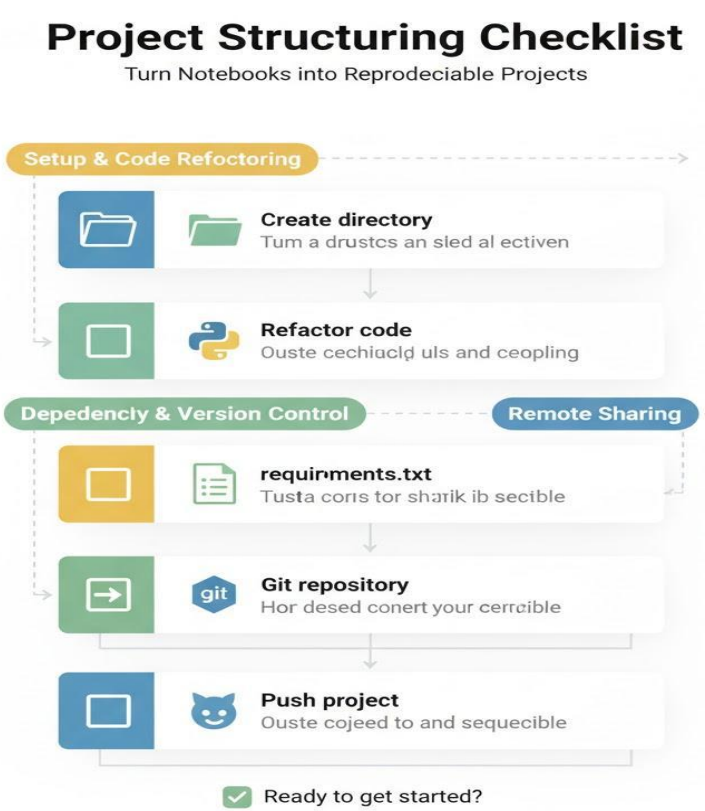
`git push`: upload from local repo to remote repo.

`git pull`: get latest code at a specific brand from remote repo.



https://education.github.com/git-cheat-sheet-education.pdf

Your Mission



Q&A



Homework

1. **Create your own repo and manage your dependencies using Poetry/uv**

2. **Read this document: Git conflict and reproduce a conflict in your code base then solve it.**

Key Takeaways

- **Notebooks are for Exploration, Scripts are for Production.**
- **Isolate Dependencies** for every project using virtual environments (venv).
- **Structure Your Code** into logical files and functions for maintainability.
- **Version Control Everything** with Git. It's your safety net and collaboration tool.
- The **add -> commit -> push** cycle is your fundamental Git workflow.

Thank you for listening!

